

삼성청년 SW·AI아카데미

B형 특강

<알림>

본 강의는 삼성청년SW·AI아카데미의 콘텐츠로
보안서약서에 의거하여
강의 내용을 어떠한 사유로도 임의로 복사, 촬영,
녹음, 복제, 보관, 전송하거나
허가 받지 않은 저장매체를
이용한 보관, 제3자에게 누설, 공개,
또는 사용하는 등의 행위를 금합니다.

Pro 연습 문제. 물류 창고

물류 창고 - 문제 이해

여러 도시를 연결하는 단방향의 도로 N개가 운송 비용과 함께 주어진다.

이 도시 중에서, 한 곳에 물류 창고를 설치하고, 총 운송 비용을 계산하고자 한다.

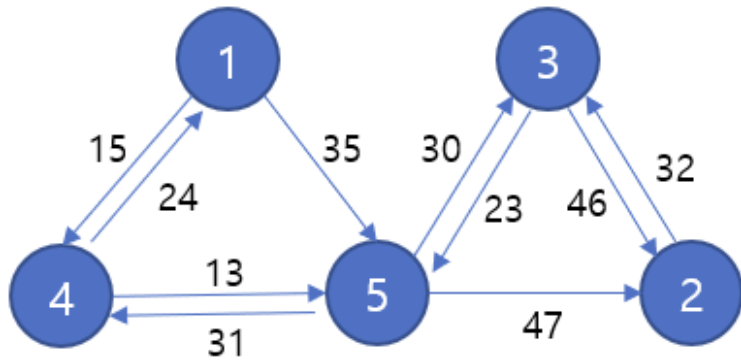
총 운송 비용은 각 도시에서 물류 창고까지 왕복에 필요한 최소 비용을 모두 합한 값이다.

물류 창고가 존재하는 도시까지의 운송 비용은 0이다.

그리고 도로는 단방향이기 때문에 허브 도시까지 가는 최소 비용과 돌아오는 최소 비용은 다를 수 있다.

예를 들어, [Fig. 1]과 같이 도시와 도로의 정보가 주어진 경우를 살펴 보자.

원 안의 숫자는 도시를 나타내는 번호이고, 화살표는 각 도시를 연결하는 단방향의 도로이고 그 위의 숫자가 운송 비용이다.



[Fig. 1]

5번 도시에 물류 창고를 설치할 경우 각 도시에서 물류 창고가 존재하는 도시까지 왕복에 필요한 최소 비용은 [Fig. 2]의 표와 같다. 따라서, 총 운송 비용은 $83 + 102 + 53 + 44 = 282$ 가 된다.

도시 #	비용	루트
1	83	1 → 4 → 5 → 4 → 1
2	102	2 → 3 → 5 → 2
3	53	3 → 5 → 3
4	44	4 → 5 → 4
5	0	

[Fig. 2]

물류 창고 - API

```
public int init(int N, int[] sCity, int[] eCity, int[] mCost)
```

각 테스트 케이스의 처음에 호출된다.

- N개의 도로 정보가 주어진다. 각 도로의 출발 도시와 도착 도시, 운송 비용이 주어진다.
- 도로 정보로 주어지는 도시의 총 개수를 반환한다.
- 단방향 도로이기 때문에 출발 도시에서 도착 도시로만 갈 수 있다.
- 출발 도시와 도착 도시의 순서쌍이 동일한 도로는 없다.
- 출발 도시와 도착 도시가 서로 같은 경우는 없다.

Parameters

- N: 도로의 개수 ($10 \leq N \leq 1400$)
- ($0 \leq i < N$)인 모든 i에 대해,
 - sCity[i]: 도로 i의 출발 도시 ($1 \leq \text{sCity}[i] \leq 1,000,000,000$)
 - eCity[i]: 도로 i의 도착 도시 ($1 \leq \text{eCity}[i] \leq 1,000,000,000$)
 - mCost[i]: 도로 i의 운송 비용 ($1 \leq \text{mCost}[i] \leq 100$)

Returns

- 도시의 총 개수를 반환한다.

물류 창고 - API

```
public void add(int sCity, int eCity, int mCost)
```

출발 도시가 sCity이고, 도착 도시가 eCity이고, 운송 비용이 mCost인 도로를 추가한다.

- init()에 없던 새로운 도시는 주어지지 않는다.
- sCity에서 eCity로 가는 도로가 이미 존재하는 경우는 입력으로 주어지지 않는다.

Parameters

- sCity: 도로 i의 출발 도시 ($1 \leq \text{sCity} \leq 1,000,000,000$)
- eCity: 도로 i의 도착 도시 ($1 \leq \text{eCity} \leq 1,000,000,000$)
- mCost: 도로 i의 운송 비용 ($1 \leq \text{mCost} \leq 100$)

물류 창고 - API

```
public int cost(int mHub)
```

mHub 도시에 물류 창고를 설치할 경우, 총 운송 비용을 계산하여 반환한다.

- mHub 도시의 운송 비용은 0으로 계산한다.
- 각 도시에서 mHub 도시까지 왕복이 불가능한 경우는 입력으로 주어지지 않는다.

Parameters

- mHub: 물류 창고를 설치할 도시 ($1 \leq \text{mHub} \leq 1,000,000,000$)

Returns

- 총 운송 비용, 다시 말해 각 도시에서 물류 창고가 존재하는 도시까지 왕복에 필요한 최소 비용을 모두 합한 값을 반환한다.

[제약사항]

1. 각 테스트 케이스 시작 시 init() 함수가 호출된다.
2. 각 테스트 케이스에서 도시의 최대 개수는 600 이하이다.
3. 각 테스트 케이스에서 모든 함수의 호출 횟수 총합은 50 이하이다.

물류 창고 - 예시

[예제]

아래의 [Table 1]과 같이 요청이 되는 경우를 살펴보자.

[Table 1]			
Order	Function	return	Figure
1	init(10, {3, 1, 5, 5, 3, 5, 1, 4, 2, 4}, {2, 4, 3, 4, 5, 2, 5, 1, 3, 5}, {46, 15, 30, 31, 23, 47, 35, 24, 32, 13})	5	Fig. 1
2	cost(5)	282	Fig. 2
3	add(5, 1, 24)		Fig. 3
4	cost(5)	251	Fig. 4
5	add(2, 1, 11)		Fig. 5
6	cost(4)	266	Fig. 6

(순서 1) 초기에 10개의 도로 정보가 주어진다.

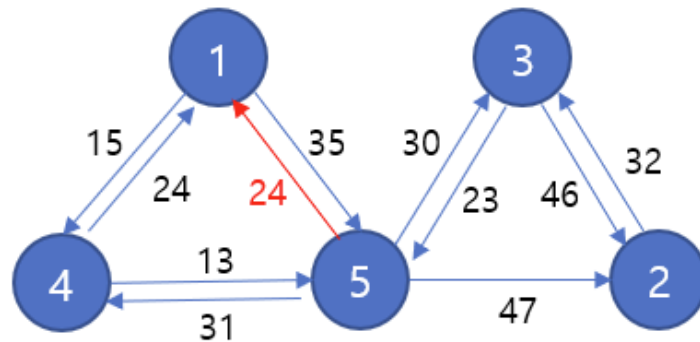
첫 번째 도로는 3번 도시에서 출발해서 2번 도시에 도착할 수 있는 도로이며 운송 비용이 46이다.

[Fig. 1]의 그림과 같이 도시들이 연결된다. 도로 정보로 주어진 도시의 총 개수 5를 반환한다.

(순서 2) 5번 도시에 물류 허브를 설치할 경우에 총 운송 비용을 계산한다.

[Fig. 2]의 표와 같이, 총 운송 비용으로 $83 + 102 + 53 + 44 = 282$ 를 반환한다.

(순서 3) 5번 도시에서 출발해서 1번 도시에 도착할 수 있는 도로가 추가된다. 해당 도로의 운송 비용은 24이다. 함수 호출의 결과는 [Fig. 3]의 그림과 같다.



[Fig. 3]

(순서 4) 5번 도시에 물류 허브를 설치할 경우에 총 운송 비용을 계산한다.

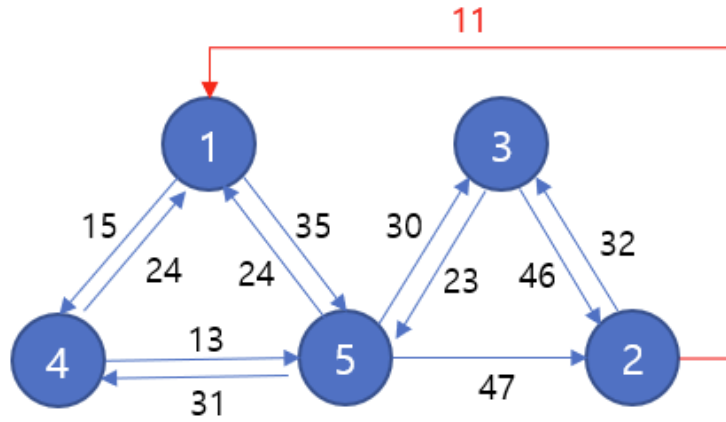
[Fig. 4]의 표와 같이, 총 운송 비용으로 $52 + 102 + 53 + 44 = 251$ 을 반환한다.

도시 #	비용	루트
1	52	1 → 4 → 5 → 1
2	102	2 → 3 → 5 → 2
3	53	3 → 5 → 3
4	44	4 → 5 → 4
5	0	

[Fig. 4]

물류 창고- 예시

(순서 5) 2번 도시에서 출발해서 1번 도시에 도착할 수 있는 도로가 추가된다. 해당 도로의 운송 비용은 11이다.
 함수 호출의 결과는 [Fig. 5]의 그림과 같다.



[Fig. 5]

(순서 6) 4번 도시에 물류 허브를 설치할 경우에 총 운송 비용을 계산한다.
 [Fig. 6]의 표와 같이, 총 운송 비용으로 $39 + 86 + 97 + 44 = 266$ 을 반환한다.

도시 #	비용	루트
1	39	1 → 4 → 1
2	86	2 → 1 → 4 → 5 → 2
3	97	3 → 5 → 4 → 5 → 3
4	0	
5	44	5 → 4 → 5

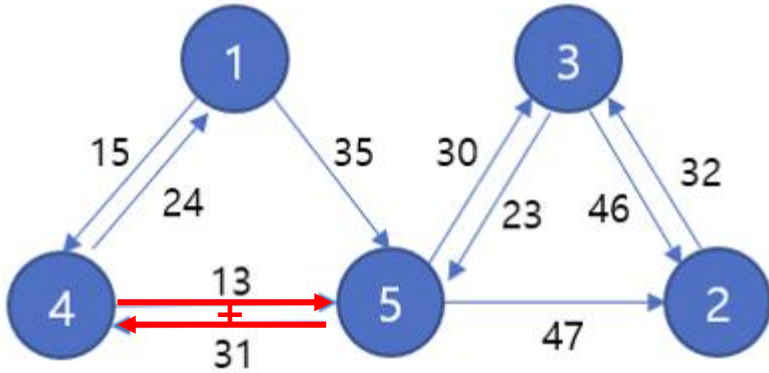
[Fig. 6]

설계하기

삼성청년SW·AI아카데미

❖ Dijkstra

- Dijkstra 문제임을 파악할 수 있는 요소들
 - 1) 가중치 그래프
 - 2) 양수의 가중치
 - 3) 목표 = 최소 비용
 - 4) 가중치의 합산에 대한 비용



도시 #	비용	루트
1	83	1 → 4 → 5 → 4 → 1
2	102	2 → 3 → 5 → 2
3	53	3 → 5 → 3
4	44	4 → 5 → 4
5	0	

cost() 계산

❖ 각 도시 → 물류 창고로 이동

```
for(int city=0; city < N; city++)  
    dijkstra(city, mHub);
```

- $O(N \times \text{Elog}V) = 8,400,000$

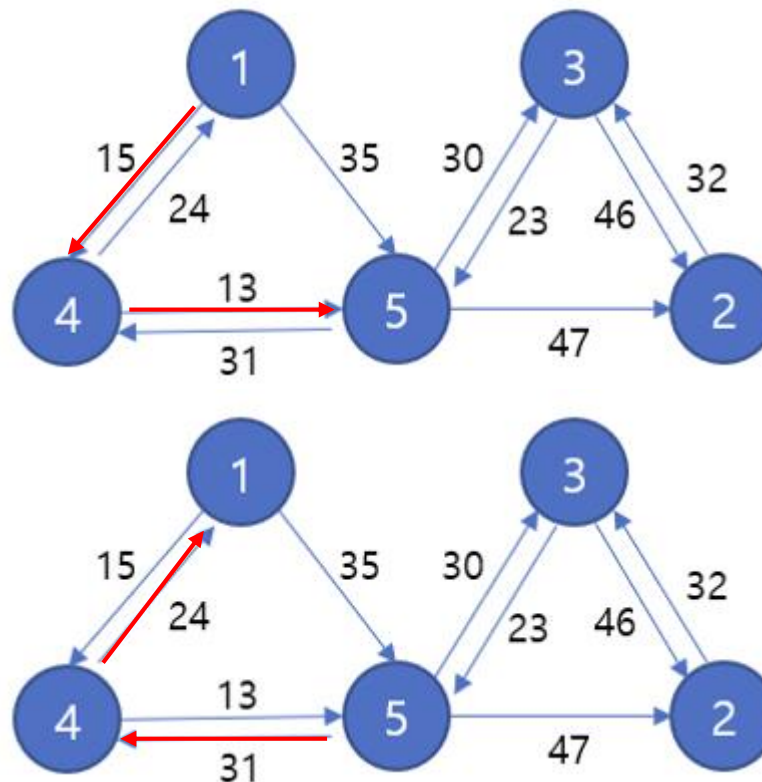
❖ 물류 창고 → 각 도시로 이동

```
dijkstra(city, mHub);
```

- $O(\text{Elog}V) = 14,000$

❖ 호출 횟수 : 50

- $50 \times (8,400,000 + 14,000) = 420,700,000$ (TLE)



도착지는 모두 동일하다

- ❖ 모든 도시에서 가려고 하는 도착지 (목적지) 는 동일하다. (mHub)
- ❖ 그리고 Dijkstra, BFS 로직에서 **출발지→도착지**의 거리는 **도착지→출발지**의 거리와 동일하다는 사실을 기억할 수 있다.
- ❖ 위 로직이 성립되지 않는 경우인데, 왜 그럴까?

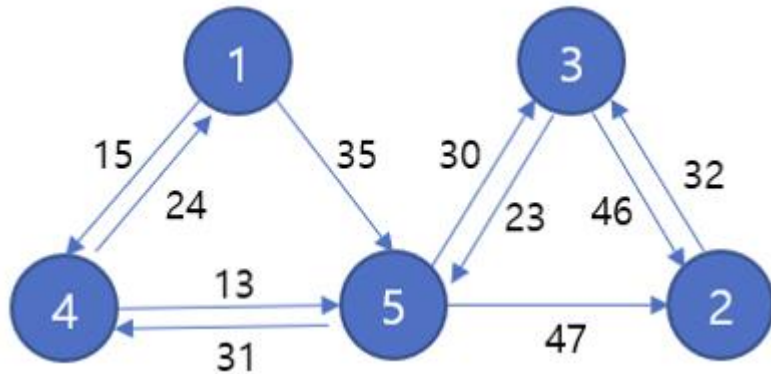
- ❖ 방향 그래프이기 때문에 출발지 $\rightarrow$ 도착지 $\neq$ 도착지 $\rightarrow$ 출발지 이다.
- ❖ 즉, 만약 **"양방향"/"무향" 그래프**였다면, dijkstra(mHub) 호출 한번으로 해결될 수 있다.
- ❖ 이 로직으로 문제를 어떻게 바꾸면 / 적용하면 문제를 효율적으로 해결할 수 있을까?

역방향 그래프 만들기

❖ 역방향 그래프를 생성하여 마치 “양방향” 처럼 동작할 수 있도록 한다.

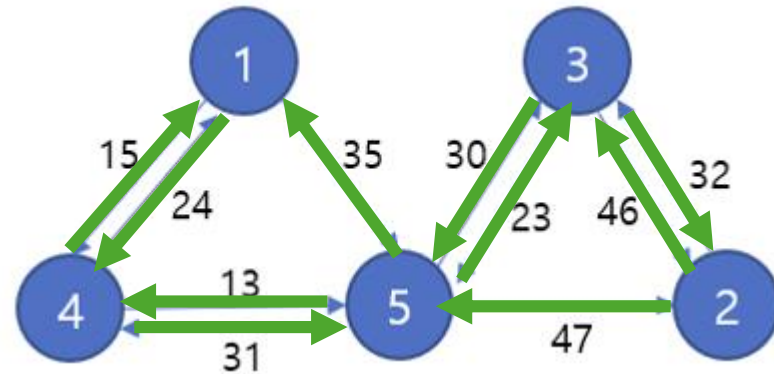
- 역방향으로 연결된 그래프를 하나 추가로 관리한다.
- 모든 도시 → 물류 창고 까지의 비용을 찾는 문제를
물류 창고 → 모든 도시 까지의 비용으로 찾는 문제로 바꾼다.

정방향 그래프



dijkstra(mHub)으로 물류창고 → 모든 도시 거리 확보 가능

역방향 그래프



dijkstra(mHub)으로 모든 도시 → 물류창고 거리 확보 가능

cost() 계산 (re)

❖ 각 도시 → 물류 창고

- `dijkstra(mHub, 역방향 그래프);`
- $O(E \log V) = 14,000$

❖ 물류 창고 → 각 도시

- `dijkstra(mHub, 정방향 그래프);`
- $O(E \log V) = 14,000$

❖ 호출 횟수 : 50

- $50 \times (14,000 + 14,000) = \mathbf{1,400,000}$

❖ init()

- 도시의 번호가 1~10억까지 → HashMap을 사용한 **id sequencing** 기법 활용 (도시 번호를 0~599로 평탄화)
- 정방향 연결을 위한 그래프 준비
- 역방향 연결을 위한 그래프 준비
- 총 도시의 개수 = idCnt개
- 시간복잡도 : **O(N)**

```
int register(int id) {  
    if(hm.get(id) == null) hm.put(id, idCnt++);  
    return hm.get(id);  
}
```

❖ add()

- 단순 간선 연결
- 시간복잡도 : **O(1)**

❖ cost()

- 정방향 연결에서 Dijkstra(mHub) → 확정되는 도시까지의 최소 비용 누적
- 역방향 연결에서 Dijkstra(mHub) → 확정되는 도시까지의 최소 비용 누적
- 시간복잡도 : **O(ElogV)**